

Gestión de cambios y versiones — CertManager

1. Control de versiones

- **Repositorio Git** (será migrado al repositorio corporativo de Claro).
- Rama principal: `main` (estado entregable).
- Versionado semántico: **MAJOR.MINOR.PATCH** (entrega actual: **1.0.0**).
- Cada cambio queda en el historial de commits (mensajes descriptivos).

2. Flujo de cambios

1. Rama de trabajo desde `main`.
2. Desarrollo + pruebas locales (`manage.py test`).
3. **Pull/Merge Request** → revisión.
4. **CI** obligatorio: chequeo de sistema, migraciones, suite de pruebas, `pip-audit`.
5. Merge a `main` solo con CI en verde.
6. Etiqueta de versión (`vX.Y.Z`) para releases.

3. Integración continua (CI)

- `.github/workflows/ci.yml` ejecuta en cada push/PR:
- `python manage.py check`
 - `python manage.py makemigrations --check --dry-run` (detecta migraciones faltantes)
 - `python manage.py test`
 - `pip-audit` (CVEs en dependencias)

4. Cambios de base de datos (migraciones)

- Todo cambio de esquema va en una **migración Django** versionada (`apps/*/migrations/`).
- El CI falla si hay modelos sin migración.
- En despliegue/upgrade: `python manage.py migrate` (en K8s, un `Job` dedicado).

5. Releases / despliegue

Paso	Acción
1	Etiquetar versión y construir el artefacto/imagen.
2	Respaldar la BD (doc 08).
3	Desplegar (Linux/Docker/K8s).

Paso	Acción
4	<code>migrate + collectstatic</code> .
5	Reiniciar servicios y validar <code>/health</code> + chequeo de prueba.
6	Registrar el cambio (versión, fecha, responsable).

6. Política de parches de seguridad

- `pip-audit` en CI detecta CVEs en dependencias; ante un hallazgo, actualizar la dependencia y re-desplegar.
- Recomendado habilitar actualizaciones automáticas de dependencias (Dependabot/equivalente).

7. Rollback

Ante una falla post-despliegue: redespregar la versión/imagen anterior y, si una migración fuese irreversible, restaurar el respaldo de BD previo (doc 08).